# Iterated Prisoner's Dilemma Machine Learning

**Scott Haakenson, Preston Korytkowski, Apar Mohabansi, Geoffrey Rajesh**

## Abstract

The Iterated Prisoner's Dilemma (IPD) problem serves as a fundamental environment for studying cooperation and competition in multi-agent systems. In this work, a variety of models will be developed to actively learn the opponent's behavior. Different architectures will be explored to see how they adapt to artificial agents and each other before being ultimately tested against human agents.

## 1 Introduction

Game Theory is seen everywhere. From board games at home with family, to war between military superpowers. Understanding how to win and how to predict an opponent's intentions can have monumental implications to how humans interact with each other and with computers. In this project, the goal is to develop a model that can simulate this situation and hope to derive a new insight into the problem.

This project revolves around the Prisoner's Dilemma, an example of game theory. In a nutshell, the Prisoner's Dilemma is a game where two parties decide to either defect (act in their own interest) or collaborate. In the case where both parties decide to collaborate, they are awarded a medium number of points each. In the case where one-party defects and the other chooses to collaborate, the one who defects are awarded many points, and the collaborator is given nothing. In the final case of both parties choosing to defect, they are both awarded a very low number of points. The aim of the game is to maximize the number of points that the player earns, regardless of the number of points that the other player earns.

The main challenge of the Iterated Prisoner's Dilemma lies in the uncertainty of the opponent's behavior, as an optimal strategy relies on an understanding of the other player's previous actions to make the best possible next action. The project addresses this by training models to predict the opponent's next action based on previous actions to make the best possible next action to maximize the long-term reward.

## 2 Related Works

Research revolving around the IPD and machine learning has been going on for many years. One such instance was by Keven Wang of the Department of Computer Science at Stanford University. In his research, the IPD was conducted, and a model was trained based on reinforcement learning (RL) to play the game. His research answered the questions of how an RL model fared against deterministic strategies, other RL agents, and a mix (Wang, 2017). While not exactly focusing on having a model perform against a real player, the research laid out by Keven Wang lays out an excellent framework for setting up the game and how an RL based model would function in the proposed scenario.

Another valuable insight on the topic was performed by Hyunsoo Park and Kyung-Joong Kim on active player modeling regarding the IPD. In the process of active learning there are three main states, the playing of the game and collecting data, the building of diverse models, and the selecting of the next action based on the uncertainty of predicting the opponents' next move (Park and Kim, 2013). This research is key to the study as a similar approach is taken, with the key difference of using different models than those present in prior art.

# 3 Dataset/Methodology

## 3.1 Iterated Prisoner's Dilemma

The IPD is a repeated version of the classic Prisoner's Dilemma game. Each player will receive the number of points shown in Table 1, depending on both players' action. In the IPD, the

| | **Cooperate** | **Defect** |
|---|---|---|
| **Cooperate** | 3, 3 | 0, 5 |
| **Defect** | 5, 0 | 1, 1 |

Table 1: Prisoner's Dilemma Score Table

game is played over multiple rounds, allowing players to develop strategies based on past interactions. In this version, the number of rounds is random, meaning players do not know in advance when the game will end. This uncertainty prevents short-term exploitation in the final rounds. Additionally, noise is introduced, meaning a player's action may be flipped with some probability. This models real-world imperfections, such as miscommunication or unintended actions.

## 3.2 Training Pipeline

This solution of the IPD involves a three-stage training pipeline: pre-training, tournament evaluation, and testing against human players.

In the pre-training phase, a model is trained with computer agents (e.g., Tit-for-Tat, Grim Trigger). This allows the model to develop an initial decision-making foundation and learn to identify basic patterns and responses to common strategies.

After pretraining, hundreds of models with different architectures and hyperparameters compete in a tournament-style evaluation to choose the top performing bots and see which architecture/hyperparameters succeed. In this tournament, models will face off against an opponent and play multiple games consisting of several rounds. The bots that finish with the highest overall score upon completion of the tournament will be considered the winners.

The top models from the tournament are then subjected to playing against real human participants. This stage of the training pipeline helps evaluate the models' adaptability against the unpredictable behavior of a human.

## 3.3 Training Data

Unlike traditional machine learning tasks that use static data, the training data is made up of algorithmic agents that follow predefined policies to interact with the models. The set of training agents includes, but is not limited to, well-known strategies such as:

- Tit-for-Tat: Cooperates on the first move and then mirrors the opponent's previous action.
- Two-Tits-for-Tat: Cooperates on the first move and defects twice in defected against.
- Tit-for-Two-Tats: Cooperates on the first move and defects if the opponent has defected twice in a row.
- Grim Trigger: Cooperates until the opponent defects once, after which it defects permanently.
- Always Cooperate/Defect: Naïve strategy.
- Random Strategy: Selects cooperation or defection with equal probability.
- Opposite: Performs the opposite of the opponent's action.
- Majority Rule: Starts with cooperation and does what the opponent has done most often.
- Pavlov: Starts with cooperation and repeats the previous move if both agent's moves are the same. Does the opposite of the previous move if both agent's moves differ.

## 3.4 Hyperparameters

Each model will be randomly assigned hyperparameters from the following list to add variation:

- Model Type: Neural network architecture. Range: RNN or LSTM. Uniform random distribution.
- Hidden Size: Number of neurons in each layer of the neural network. Range: 8 – 32 Neurons. Uniform random distribution
- Number of Episodes: Number of training iterations through all selected agents.

Similar to the number of epochs. Range: 20 – 50. Uniform random distribution.

- Number of Agents / Training Agents: 3 – 8 agents selected from the 10 classic strategies. Uniform random distribution.
- Training Learning Rate: Controls the gradient descent step size during the initial training against agents. Range: 0.005 – 0.05. Log-uniform random distribution.
- Online Learning Rate: Controls the gradient descent step size while online learning. Range: 0.05 – 0.2. Log-unform random distribution.
- Epsilon: Probability of taking random actions during training. This encourages exploration of diverse strategies. Range: 0.05 – 0.2. Uniform random distribution.
- Entropy Coefficient: Probability of taking random actions during online learning. This encourages exploration of diverse strategies while facing other models. Range: 0.001 – 0.01. Log-uniform random distribution.
- Memory Size: Determines how many previous interactions are considered during online learning. Range: 10 – 20. Uniform random distribution.

### 3.5 Human Agent

Another crucial training agent is that of the human agent. By having a human player provide real-time input, the model can learn from authentic human behavior that may not be able to be modeled in an algorithm. Human agents help the different models to recognize and adapt to different, unpredictable behaviors. This is crucial to include as the model will need to play against a real person in the future so training it now on real behavior will improve how it will perform.

### 3.6 Recurrent Neural Networks

The recurrent neural network model will allow for the model to remember prior information from previous iterations of game play. In pre-training, this will allow it to recognize certain patterns over time and react accordingly. In fine-tuning, the model will be able to still recognize these patterns, but also can tell if the human changes patterns, allowing the model to adjust its behavior.

### 3.7 Implementation of RNN

To investigate the interaction of an RNN in the iterated prisoner's dilemma, an RNN agent from the PyTorch library is incorporated into the project. The RNN has an input layer that takes in both its own previous action and its opponent's previous action. The model also consists of two stacked layers: a layer to process the input states and a layer mapping to a hidden state. The model has a fully connected (FC) layer, taking outputs from previous layers to make a final decision.

In the project's Gym environment, the RNN model interacts and learns from the different computer agents listed in Section 3.2. The model's hidden state is reset, and it is possible to assume that both the model and the agent will cooperate at the start of each episode of training. Then throughout each round, the RNN uses the previous states (the previous moves from itself and its opponent) to calculate the current state. This state is then passed to the hidden state and the RNN produces a probability distribution to decide between cooperating and defecting against the agent. The present implementation uses an epsilon-greedy exploration approach, with an epsilon value of 0.1, meaning that 90% of the time the model will follow the probability distribution and the other 10% of the time the model's action will be completely random.

### 3.8 LSTM

Like recurrent neural networks, long short-term memory has a memory of previous information. However, this memory is more advanced than that of recurrent neural networks as it is composed of three gates to control learning: an input gate, an output gate, and a forget gate. Together, these gates allow the model to decide which new information to store, which old information is no longer needed and can be discarded, and what parts of memory should be used in its next move. The model will be able to learn more complex patterns and have different memory patterns relating to different behaviors from the opponent. In fine-tuning, this model will also be better at handling inconsistencies in the human's strategy.

### 3.9 Proposed Evaluation

The evaluation of the models will be divided into two distinct halves. The first half consists of purely of the iterated prisoner's dilemma being played while the other half consists of a tournament styled evaluation.

The first half is seen in the code file of "Game.py". After being trained with an agent, the model plays against another agent. The actions of the model are reported, and an overall score is stated at the end of the set number of rounds. For this simple evaluation, a visual confirmation is used to confirm if the actions of the model are consistent with its training agent.

The second half of the evaluation will be executed in a proposed tournament style, where multiple models compete over repeated games to simulate human agents. A set of models will be trained during the pre-training stage with varying parameters and hyperparameters. These models will then compete against each other to accumulate score over the entire tournament. Each game, the models will fine-tune themselves to the opponent to simulate a human agent game. However, each game is independent, and modified parameters will not carry over into new games. This evaluation period is used to improve the performance of models over a longer period and actions.

### 3.10 Final Human Interaction

After evaluation, the best models will be selected and challenge human agents. This stage will be executed the same way as the evaluation in Section 3.8. This can be used to see if the models can outperform humans at the IPD game. In traditional machine learning, this stage would be most like the test data evaluation.

### 3.11 Detailed RNN Training Process

The RNN model is trained against bot agents within the Gym environment. Here, the RNN loops through several episodes (initialized to 200), loops through each agent designated to play against (ranging from 1 to 6 agents available to play against), and plays 50 rounds per episode against each agent.

At the start of each episode, the RNN resets its model state and its hidden layer so that each episode is independent. Then during each round, the model obtains its own and the agent's actions from the round prior (if a prior round exists) to set its state. During the first round, the model sets its state assuming that both itself and the agent will cooperate. A probability distribution is then formed for the RNN's action based on its state. Most of the time, the model's action will come directly from this probability distribution. However, due to the Epsilon-Greedy exploration approach, there is a chance (10% in this case) that the RNN will choose an action completely randomly. This approach allows the model to optimize its strategies while also exploring and trying new actions to avoid converging to a poor strategy.

After both the model and the agent have chosen their actions, the reward is calculated based on the standard rules of the Prisoner's Dilemma. This reward is then added to the model's list of rewards, and then the next round commences.

After all rounds within an episode are completed, using a discount factor of 0.9, the model then calculates the discounted returns for each reward from each round of the episode, allowing it to optimize long-term gains rather than prioritizing short-term rewards.

Finally, at the end of the episode, the model calculates the policy loss using the equation below:

$$Loss = -\sum_i \log\_probs[i] * (discounted\_returns[i] - baseline) \quad (1)$$

Where log_probs[i] is the log probability of the action taken by the RNN at round "i" and baseline is the average of all discounted returns (to help reduce variance). With this policy loss, PyTorch can compute the gradients for all the RNN's parameters using the ".backward()" function. The Adam Optimizer then uses these gradients to update the model parameters to increase the expected reward.

The RNN continues this process for every episode. Upon completion, the RNN model should have its parameters updated enough to where it will make optimal decisions against the

agent(s) it was trained against and generalize these strategies to different agents or even human players.

### 3.12 Initial Experimental Results

In the initial experimentation, the RNN model was worked with and the first half of the evaluation process as noted in Section 3.9 was followed through with. This model is trained in the Gym environment with different combinations of the agents listed in Section 3.3. In the figure below, you can see the model's progression as it iterates through its training episodes against the Tit-for-Tat agent.

```
Pre-training bot in Gym...
Episode 0, Cumulative Reward: 107, Avg Model Action: 0.44, Avg Agent Action: 0.44
Episode 20, Cumulative Reward: 148, Avg Model Action: 0.96, Avg Agent Action: 0.96
Episode 40, Cumulative Reward: 147, Avg Model Action: 0.88, Avg Agent Action: 0.9
Episode 60, Cumulative Reward: 149, Avg Model Action: 0.98, Avg Agent Action: 0.98
Episode 80, Cumulative Reward: 144, Avg Model Action: 0.88, Avg Agent Action: 0.88
Episode 100, Cumulative Reward: 143, Avg Model Action: 0.86, Avg Agent Action: 0.86
Episode 120, Cumulative Reward: 147, Avg Model Action: 0.94, Avg Agent Action: 0.94
Episode 140, Cumulative Reward: 148, Avg Model Action: 0.96, Avg Agent Action: 0.96
Episode 160, Cumulative Reward: 149, Avg Model Action: 0.98, Avg Agent Action: 0.98
Episode 180, Cumulative Reward: 150, Avg Model Action: 1.0, Avg Agent Action: 1.0
Pre-training complete!
```

Figure 1: RNN Pre-Training vs. Tit-for-Tat

Figure 1 illustrates how the RNN model's average action grew from 0.44 all the way to 1.0 by the final round. With 0 representing defection and 1 representing cooperation, it shows that in Episode 0, this model was close to even with choosing its move but was leaning with a slight bias toward defection. By Episode 180, the RNN's average action was 1.0, meaning that it learned to cooperate every move. This is the optimal strategy against a Tit-for-Tat agent, as is shown with a Cumulative Reward of 150, the highest possible score in a round of this length against Tit-for-Tat.

## 4 Results

### 4.1 Winning Models

500 bots were created with varying architectures and hyperparameters regarding model type, hidden size, memory size, learning rate, and what agents it was trained on. Then, these bots were subjected to the tournament section of the training pipeline, where each bot played multiple IPD games against other bots. In the tournament, the maximum number of cumulative points possible for a bot was 5000. Each bot's cumulative score was tracked, and in the figure below you can see the scores of the top performing bots as well as some information regarding hyperparameters of the models.

| Rank | Bot ID | Model | Avg. Score per Round | Hidden Size | # of Agents Trained on |
|---|---|---|---|---|---|
| 1 | Bot_482 | RNN | 2.42 | 14 | 7 |
| 2 | Bot_404 | LSTM | 2.25 | 22 | 8 |
| 3 | Bot_318 | LSTM | 2.19 | 27 | 7 |
| 4 | Bot_136 | LSTM | 2.16 | 25 | 8 |
| 5 | Bot_38 | LSTM | 2.16 | 19 | 6 |

Figure 2: Top Performing Models

Initially, one can see that LSTMs appear to be the top performing model architecture, as 4 of the top 5 are LSTMs. However, the top performing model was an RNN with an average round score of 2.42.

### 4.2 Strategy Balance Index

The strategy balance index of a model is a calculation derived from the model's value of (cooperation+foregiveness-retaliation)/3. The value tells us about the behavioral tendencies of the model while playing the IPD. A higher strategy balance index value implies that the model is more cooperative and less retaliatory, while a lower value implies that the model is more aggressive and likely to defect.

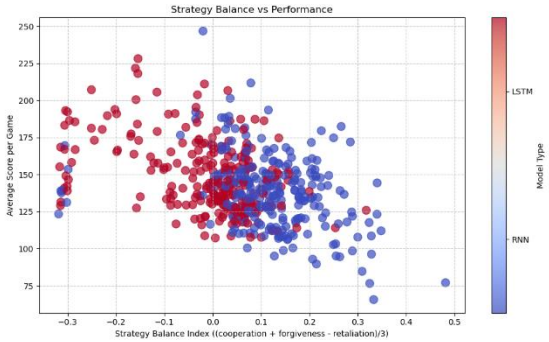


Figure 3: Strategy Balance vs Performance Results

Figure 2 shows how LSTM and RNN models clustered around different strategy balance index values. LSTMs clustered towards much lower and negative strategy balance index values but also achieved higher average scores per game. Meanwhile, RNNs clustered towards larger strategy balance index values but achieved lower average scores per game on average. These trends show that LSTMs lean towards more cooperative strategies, while RNNs go for more aggressive strategies.

### 4.3 Individual Strategy Analysis

The models that were the most successful were found to have similar characteristics. A negative correlation was found between cooperation and performance. A positive correlation was found

between retaliation and performance. A weak negative correlation was found between forgiveness and performance.

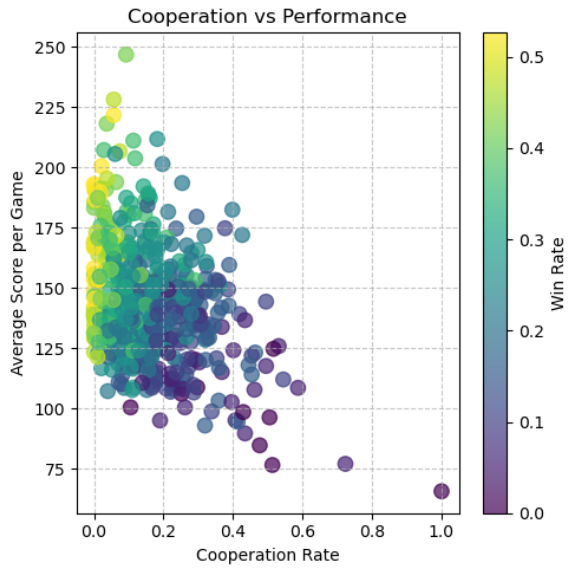


Figure 4: Cooperation vs. Performance Results

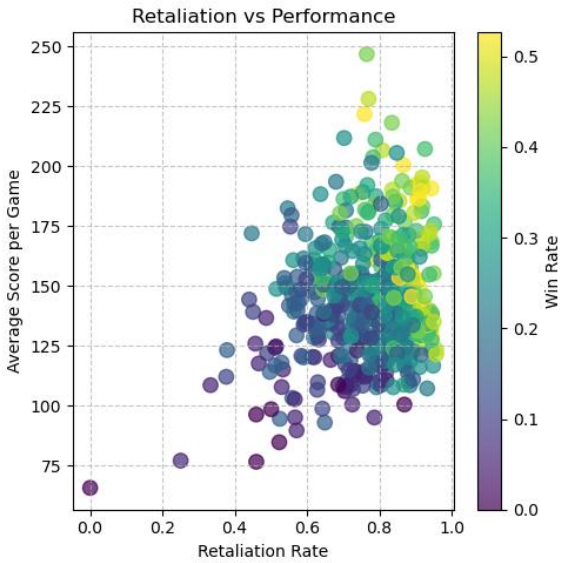


Figure 5: Retaliation vs Performance Results

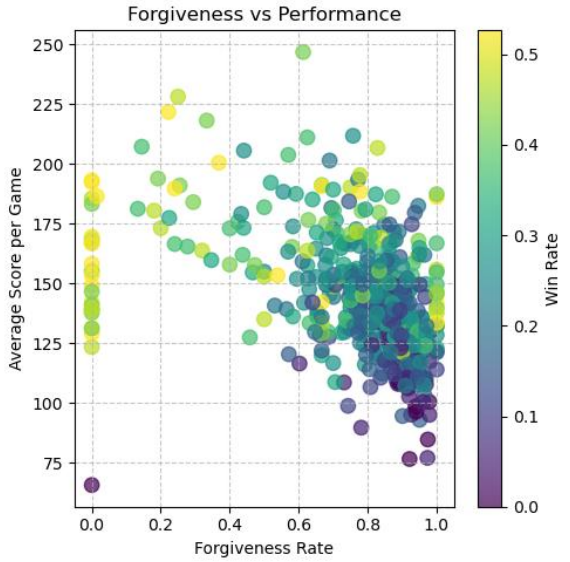


Figure 6: Forgiveness vs performance Results

This makes sense because cooperation is only beneficial when both sides cooperate. And when the other side cooperates, it's immediately beneficial for the model to defect. This ultimately has pushed the optimal strategy to be more retaliatory and less cooperative. It can also be seen that forgiveness is also a negative correlation as the most forgiving models were simply exploited by the other side. To prevent exploitation, the models had to become less forgiving.

### 4.4 Model Architecture

LSTMs performed better on average, but performance seems very similar between model types. A lower learning rate appeared to be optimal as it allows the models to remember more from previous trials.

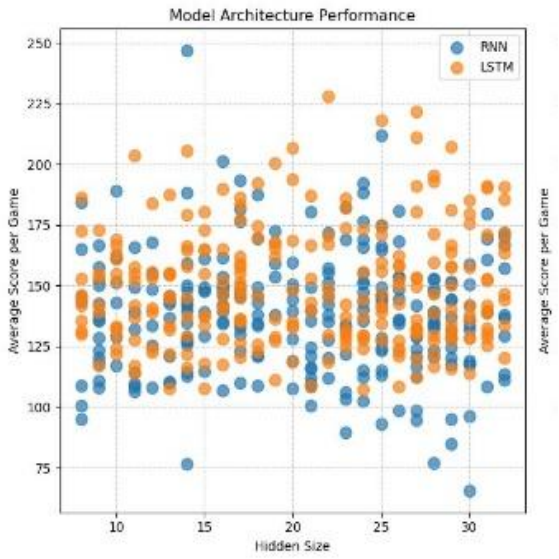

Figure 7: Model Architecture Performance

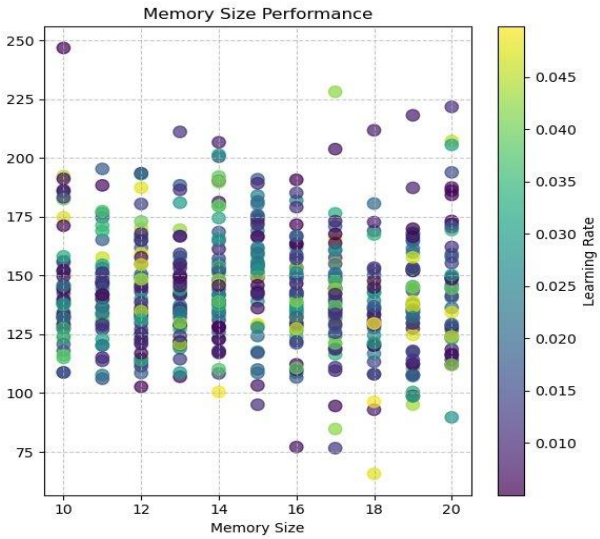


Figure 8: Memory Size Performance

## 4.5 Agent Training Impact

As discussed earlier, the models were trained on various strategies that ranged from encouraging more aggressive play to more cooperative play.

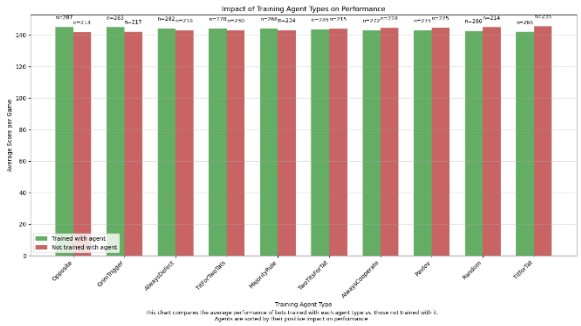


Figure 9: Impact on Training Agent Types on Performance

As shown in Figure 9, when a model is trained with an agent that promotes a nastier behavior relative to its other agents, the model tends to perform better than had it not been trained on the agent. On the other side of the spectrum, when a model is trained with an agent that promotes more cooperative play, they end up performing worse than had they not been trained with the agent at all. These trends likely occur as nastier models promote defection while the opposite promote cooperation, thus leading to situations where models both take advantage and are taken advantage of.

## 4.6 Other Hyperparameters

The number of training agents each model was subjected to in pre-training differed across the 500 total models. Upon evaluation, it was found that there appeared to be no correlation between the number of agents the model was trained on and the performance of the model. This is displayed in the figure below.

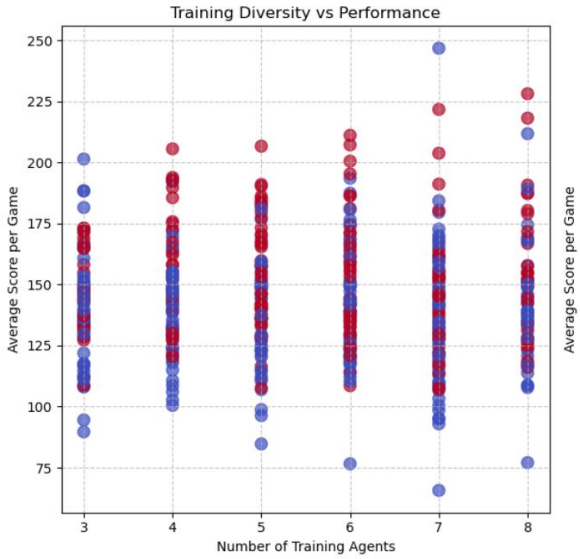


Figure 10: Training Diversity vs. Performance

Two other hyperparameters, epsilon and entropy, were related to the models' exploration rate in pre-training and online learning respectively. The differing of these hyperparameters also showed to have no correlation with the models' performance. This is displayed in the figure below.

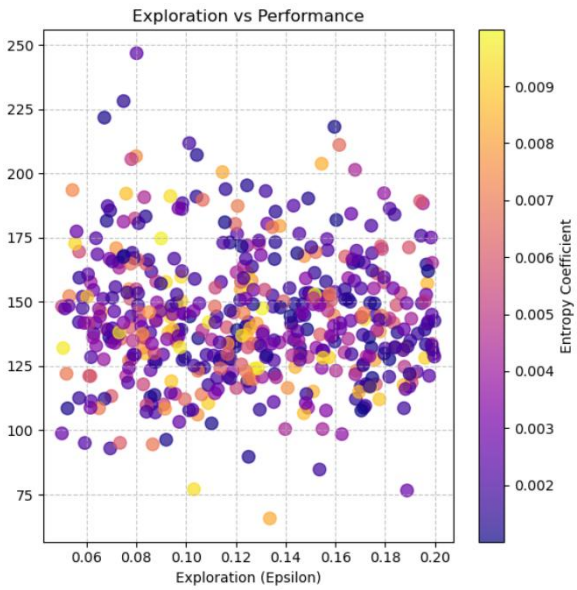


Figure 11: Exploration vs. Performance

## 4.7 Human Evaluation

As was noted in earlier sections, the top performing models in the tournament evaluation tended to have nasty behavior. These models often defected

immediately and attempted to exploit their opponents to gain a maximum number of points. This behavior was very evident as these top models were subjected to live play of the IPD against a real-life human agent. In the live play, these models continued their nasty behavior. Cooperation from the models was only seen to be developed upon retaliation followed by forgiveness.

## 5 Future Work

While this project has successfully explored how RNN and LSTM-based agents perform when confronted with the IPD, there is still room for growth and more exploration around the topic.

One such avenue for work would be to investigate how other models perform as well. An example of this would be Q-Learning, which has the potential to develop more nuanced traits through trial and error. This model has the potential to perform better than an RNN or an LSTM could as well as other models for future research.

While training the models, it was often found that when the model falls into a state when it finds defecting to be the move favorable action to take, it would continuously do so without much of a real reason to cooperate. This behavior is likely caused by the model attempting to maximize its own score, regardless of what its opponents' own score is and its training weights/bias. In the future, modifying the initial training weights to help encourage cooperation could be an interesting direction to explore the IPD, to see what it takes for cooperation to occur.

Another useful addition to this project would be to add noise while training the models. While somewhat counterintuitive at first, the addition of noise helps to mimic real world conditions as it is used to represent misinterpreted actions.

Lastly, there are two key improvements to the tournament and its class that can be made. The first of which is during the progression of the tournament, to remove low performers and duplicate the best one. This form of model evolution mimics the idea of evolutionary game theory, as strategies can emerge and become adapted. Another key improvement would add a sort of tier system to the tournament, as the best performers compete against other high performing models only.

## 6 Conclusion

In this project, the Iterated Prisoner's Dilemma was explored to test decision-making and adaptation using machine learning. Recurrent neural networks and long short-term memory recurrent neural networks were utilized to create hundreds of agents capable of learning and adapting. A set of training agents using well-known IPD strategies were leveraged to give the models an initial decision-making foundation. By differentiating which and how many of these training agents a model was trained on, as well as differing the architecture and hyperparameters such as learning rate and hidden size, a diverse set of models were created and were able to be tested against one another. This allowed us to gain an understanding of which architecture and hyperparameters result in better decision-making and adaptability.

The results of the tournament showed that LSTM models, on average, outperformed RNN models. LSTMs showed a nastier, less forgiving approach whereas RNN models tended to be more cooperative and more forgiving. A positive correlation between retaliation and performance was discovered, and, conversely, a negative correlation was found between cooperation and performance. This explains why LSTMs tended to outperform RNNs. Also, a weak negative correlation was found between forgiveness and performance, further explaining LSTMs better performance.

The work of this project demonstrates how machine learning can be utilized to gain a further understanding of classical game theory. While the adaptability of RNNs and LSTMs was successfully seen, further exploration should be conducted to see how other model architectures would perform, such as Q-Learning models. Exploring cooperation-encouraged approaches could help gain a further understanding of the IPD given the nasty nature of most of the top performing models in this project. Cooperation may better align with human objectives, allowing the results of the experiment to be more applicable and generalized to the real world.

# 7 Researcher Biographies

## 7.1 Scott Haakenson

NetID: haakens3
Department: Computer Science
Expected Graduation: May 2026

## 7.2 Preston Korytkowski

NetID: korytko2
Department: Computer Science
Expected Graduation: December 2025

## 7.3 Apar Mohabansi

NetID: mohabans
Department: Computer Science
Expected Graduation: May 2026

## 7.4 Geoffrey Rajesh

NetID: rajeshge
Department: Electrical and Computer Engineering
Expected Graduation: May 2025

## References

Keven Wang. 2017. *Iterated Prisoners Dilemma with Reinforcement Learning*. Department of Computer Science, Stanford University.

Hyunsoo Park and Kyung-Joong Kim. 2013. *Active Player Modeling in the Iterated Prisoner's Dilemma.* Hindawi Publishing Corporation.